

UDP & 과제1 보고서

로봇 21기 예비단원 최윤서

<과제1 보고서>

1. 과제 이해

- 3인 1조로 서로 대화를 주고받은 문자(카카오톡) 구현
- 쌍방 통신 가능해야함

2. UDP 구현

2-1. 상대방 IP, 내 IP 저장

```
QHostAddress ROBIT_IP = QHostAddress("172.100.0.183");
QHostAddress ROBOT_IP = QHostAddress("172.100.6.53");

uint16_t TEXT_PORT = 10004;
```

- ROBIT_IP = 내 IP
- ROBOT_IP = 상대방 IP
- TEXT_PORT는 상대방과 통일해야 통신됨

2-2. Connect

- 헤더파일에 text_socket 동적 메모리 할당

- 예제 파일과 동일하게 작성

```
qDebug() << "1. 프로그램 시작";
if(text_socket->bind(QHostAddress::Any, TEXT_PORT, QUdpSocket::ShareAddress))
{
    qDebug() << "2. bind 성공";

    connect(text_socket, &QUdpSocket::readyRead, this, &MainWindow::udp_read);

    qDebug() << "3. connect 성공";
} else {
    qDebug() << "bind 에러" << text_socket->errorString();
}
```

- 중간중간 작동 확인 코드 추가

2-3. Read

- QByteArray 형의 데이터 변수를 그대로 받으면
일반적인 텍스트로 출력 안됨
- 받은 바이트 데이터를 UTF-8로 변환해야함

```
// QByteArray를 QString(텍스트)으로 변환
QString message = QString::fromUtf8(data);
```

- 상대방이 보낸 문자 출력

```
// UI에 상대방 정보와 메시지 출력
ui->plainTextEdit->appendPlainText(QString("%1")
                                     .arg(message));
```

2-4. Write

- 처음엔 예제파일의 write함수부터 시작
- 개발 중 send 버튼이랑 연동시켜서 write 함수 대체
- 입력창의 내용 가져오기

```
QString message = ui->plainTextEdit->toPlainText();
```

- toPlainText() : QPlainTextEdit에 입력되어있는 텍스트를 QString으로 가져옴
- 만약 입력창에 Hello Robot을 입력하면 message에 "hello robot"이 저장됨
- QString -> QByteArray로 변환

```
QByteArray data = message.toUtf8();
```

- QString을 UTF-8 형식의 바이트 데이터로 변환
- 네트워크로 데이터를 전송할 때는 문자열 자체보다는 바이트 데이터가 사용됨
- 따라서 message.toUtf8()을 사용해야함

- UDP 데이터 전송

```
qint64 sentBytes = text_socket->writeDatagram(data, ROBOT_IP, TEXT_PORT);
```

- 실제로 UDP 데이터를 보내는 핵심 부분
- text_socket: 앞에서 생성한 QUdpSocket의 객체
- writeDatagram: UDP 데이터 전송 함수
-> 데이터, 목적지 IP, 목적지 포트 순으로 입력함
- 목적지 IP가 ROBOT_IP이므로 상대방 IP를 입력함

- 전송 결과 확인

```
if (sentBytes == -1) {
    qDebug() << "전송 실패 에러:" << text_socket->errorString();
}
```

- writeDatagram은 전송 결과를 qint64로 반환

3. 기능

- 상대방이 보낸 문자는 왼쪽에 정렬
- 내가 보낸 문자는 오른쪽에 정렬
- 버튼 누를 때마다 노란색으로 이펙트 발생
- 상대방이 보낸 문자와 내가 보낸 문자 구별 가능

<UDP 보고서>

1. 서론

1-1. 통신 프로토콜의 필요성

- 프로토콜: 컴퓨터 네트워크에서 데이터를 주고받을 때 수행되는 절차
상호 연동되는 시스템이 전송 매체를 통해 데이터를 주고받을 때 따르는 표준화된 규칙
- 컴퓨터 상의 응용 프로그램들은 프로토콜을 통해 서로 통신
- 따라서 서로의 프로그램은 서로 이해할 수 있는 규범인 프로토콜이 필요함
- 예를들어 7비트를 보내는 컴퓨터가 8비트를 사용하는 컴퓨터에게 데이터를

전송하려고할 때 그대로 보내면 제대로 된 정보를 받을 수 없음

- 따라서 7비트를 8비트로 변환시키는 프로토콜이 필요함

1-2. 컴퓨터 네트워크와 데이터 통신의 개념

1-2-1. 컴퓨터 네트워크

- 독립된 장치들이 서로 데이터를 교환하도록 물리적 또는 무선 기술로 묶인 체계
- 구성요소: 종단시스템(사용자가 쓰는 PC), 중개시스템(데이터전달 라우터,스위치 등)

1-2-2. 데이터 통신

- 두 장치 간에 디지털 정보를 주고받는 기반 기술
- 기술이 발전함에 따라 고속화, 대용량화, 모바일화라는 특징을 지님

2. 네트워크 통신의 기본 개념

2-1. 네트워크 통신의 기본 구조

- 노드: 정보를 주고받는 기기(컴퓨터, 스마트폰)
- 간선: 노드와 노드를 연결하는 유무선 통신 매체
- 메시지: 노드 간에 주고받는 데이터 또는 정보
- 프로토콜: 데이터를 안전하게 주고받기 위해 정한 통신 규칙

2-2. 클라이언트와 서버

2-2-1. 서버

- 어떠한 서비스를 제공하는 호스트
- 어떠한 서비스란? 파일, 웹페이지, 메일 등등

2-2-2. 클라이언트

- 서버에게 어떠한 서비스를 요청하고 서버의 응답을 제공 받음
- 식당에서 종업원과 손님의 관계를 생각하면됨

2-3. IP주소와 포트 번호

2-3-1. IP 주소란?

- Internet Protocol의 약자
- 인터넷 상에서 사용하는 주소 체계
- 인터넷에 연결된 모든 PC는 IP 주소 체계를 따라 네 덩이 숫자로 구분됨
- 네 덩이의 숫자로 된 주소 체계를 IPv4

2-3-2. 터미널에서 도메인 IP 확인하기

- 터미널을 열고 nslookup [naver.com](https://www.naver.com) 을 입력하면 됨
- ubuntu 환경에서는 ip addr 을 입력하면 됨

2-3-3. IPv4

- IP 주소 체계의 네번째 버전을 뜻함
- 각 덩어리마다 0~255 사이의 숫자로 표현 가능
- 이 시스템으로는 약 43억개의 IP주소를 나타낼 수 있음
- IP 주소 중 이미 몇가지는 용도가 정해져 있음

2-3-4. 용도가 정해져있는 IP 주소

- localhost, 127.0.0.1: 현재 사용중인 로컬 PC
- 0,0,0,0 / 255.255.255.255: broadcast address -> 로컬 네트워크에 접속된 모든 장치와 소통하는 주소
- 서버에서 접근 가능한 IP 주소를 broadcast address로 지정 -> 모든 기기에서 접속 가능

2-3-5. PORT란?

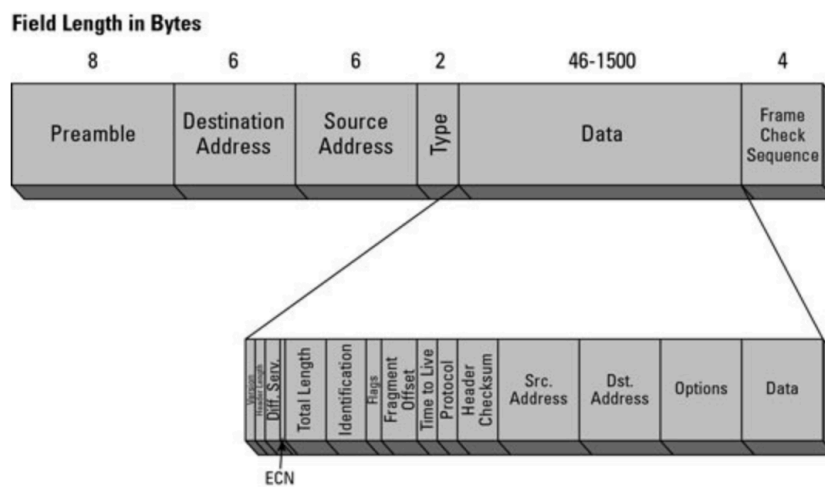
- IP 내에서 애플리케이션 상호 구분을 위해 사용하는 번호
- 포트 숫자: IP 주소가 가리키는 PC에 접속할 수 있는 통로
- 이미 사용중인 포트는 중복해서 사용할 수 없음
- 0~65535 까지 포트 번호 사용 가능
- 그 중에서 0~1024번까지 포트 번호는 주요 통신을 위한 규약에 따라 이미 정해짐

- 주요 포트: 22 -> ssh / 80 -> HTTP / 443 -> HTTPS

2-4. 패킷(Packet)이란?

2-4-1. 패킷의 정의

- 정보 기술에서 패킷 방식의 컴퓨터 네트워크가 전달하는 데이터의 형식화된 블록
- 컴퓨터 네트워크에서 데이터를 주고받을 때 정해놓은 규칙
- 즉, 정보를 보낼 때 특정 형태를 맞춰서 보내는 형식



2-4-2. 패킷의 구조

- IP 패킷은 헤더로 정의됨
- 해당 헤더는 많은 필드가 포함됨

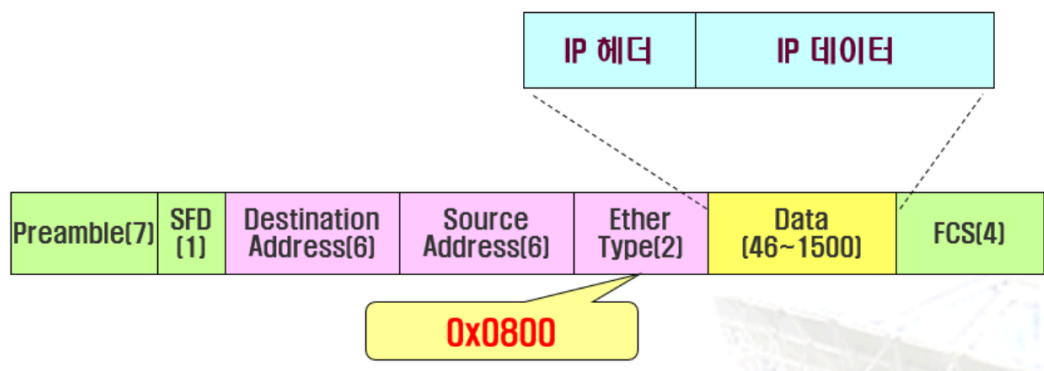
2-4-2-1. 헤더에 있는 요소

- 1) 버전: 사용 중인 IP 식별에 사용
- 2) TTL: 패킷이 네트워크에 남아있을 수 있는 시간
- 3) 프로토콜: IP 패킷의 데이터 부분이 전달되는 전송 계층 프로토콜
- 4) 소스 주소: 패킷을 네트워크로 보내는 장치의 IP 주소
- 5) 대상 주소: 패킷이 전송되는 주소

2-5. 데이터그램(Datagram)이란?

2-5-1. 데이터그램이란?

- 인터넷을 통해 전달되는 정보의 기본 단위
- IP 계층의 가변길이 패킷
- 헤더와 데이터 부분으로 구성
- 헤더는 20바이트~60 바이트
- 라우팅과 전달에 필요한 정보를 포함
- TCP/IP에서는 헤더를 4바이트 단위로 유지



IP 데이터그램의 캡슐화

0										1										2										3	
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
Version				HLEN				Type of Service(1)						Total Length(2)																	
Identifier(2)										O D M			Fragment Offset																		
Time of Live(1)					Protocol(1)					Header Checksum(2)																					
Source Address(4)																															
Destination Address(4)																															
Option + Padding(0~40)																															

20
byte

데이터그램과 헤더 형식

2-6. 유니캐스트

- 네트워크 상에서 하나의 출발지와 하나의 수신지가 1:1로 데이터를 전송하는 가장 기본적인 통신 방식
- 정보를 전송하기 위한 프레임에 자신의 MAC 주소와 목적지의 MAC 주소를 첨부하여 전송하는 방식

**MAC 주소란? 컴퓨터나 스마트폰 같은 네트워크 장비의 랜카드(NIC)에 부여된 물리적이고 고유한 식별 주소
- 유니캐스트로 데이터 전송? -> 같은 네트워크에 있는 모든 시스템이 MAC 주소를 받음 -> 각자 자신의 MAC 주소와 비교
-> 맞지않다면 프레임 버림 / 맞다면 프레임 받아서 처리
- 한개의 목적지 MAC 주소를 사용하고 CPU 성능에 문제 주지 않음

2-7. 브로드캐스트

- 로컬 네트워크에 연결되어있는 모든 시스템에게 프레임을 보내는 방식
- 브로드캐스트용 주소 미리 정해져있음
- 수신받는 시스템은 이 주소가 오면 패킷을 자신의 CPU로 전송
-> CPU가 패킷을 처리
- 모든 시스템으로 패킷 전송 -> 트래픽 증가, CPU 성능 저하 생김
- 시스템의 MAC 주소를 모르는 경우, 네트워크에 있는 모든 시스템에게 알리는 경우, 라우터끼리 정보를 교환하는 경우에 사용됨

2-8. 멀티캐스트

- 네트워크에 연결되어있는 시스템 중 일부에게만 정보 전송하는 방식
- 특정 그룹에 속해있는 시스템에게만 한 번에 정보를 전송할 수 있는 방법
- 라우터가 멀티캐스트를 지원해야만 사용 가능함
- 그룹 단위로 묶어 그 그룹의 Host들은 동시에 데이터를 받을 수 있음
- UDP를 사용하여 전송 -> 신뢰성은 보장 X
- 서버가 멀티캐스트 주소로 데이터를 전송 중일 때 중간에 클라이언트가 끼어들면 처음부터 데이터 받을 수 없고 중간부터 받음
- 하나의 클라이언트에서 여러 멀티캐스트 주소를 수용 가능

- 클라이언트에서 멀티캐스트 사용하는 어플리케이션 시작 -> 멀티캐스트 IP 주소와 멀티캐스트 MAC 주소를 라우터에 등록함으로 멀티캐스트 그룹에 등록됨

3. OSI 7계층과 TCP 모델

3-1. OSI 7계층이란??

- 컴퓨터 네트워크는 외형상 호스트 시스템 + 전송매체로 구분
- 국제 표준화 단체인 ISO에서 복잡한 네트워크 기능을 서로 연관있는 그룹끼리 묶어 7계층으로 분할한 모델
- 7계층으로는 물리, 데이터 링크, 네트워크, 전송, 세션, 표현, 응용 계층이 있음
- 물리 계층이 가장 하위 계층이며 응용 계층이 가장 상단 계층임

3-2. 각 계층의 역할

3-2-1. 물리 계층

- OSI의 1계층
- 물리적 매체-> 비트 흐름을 전송하기 위한 필요한 기능을 조정
- 호스트를 전송 매체와 연결하기 위한 인터페이스 규칙과 전송 매체의 특성을 다룸

3-2-2. 데이터 링크 계층

- OSI 2계층
- 중계 시스템 간의 데이터 전송을 원활하게 하여 오류 검출과 회복 기능을 제공
- 정확한 순서에 따라 인접한 노드 간의 데이터 전송 실행

3-2-3. 네트워크 계층

- OSI 3계층
- 응용 프로세스 간의 데이터 전송 시 정보의 경로 선택과 중계 기능 수행
- 데이터 전송 시 각종 네트워크 이용하게 됨
-> 이 때 네트워크 내부나 다른 네트워크 사이 중계

3-2-4. 전송 계층

- OSI 4계층

- 정보를 상대방에게 보내는 운반, 운송 기능 관리
- 송신 프로세스와 수신 프로세스 간의 연결 기능을 제공 -> 안전한 데이터 전송 지원

3-2-5. 세션 계층

- OSI 5계층
- 전송 계층에서 제공하는 통신로 사용 -> 프로세스 간 대화 성립시킴
- 대량의 데이터를 보내는 과정에서 필요한 동기점을 정함
- 전송 계층의 연결과 비슷하지만 더 상위의 논리적 연결을 지원

3-2-6. 표현 계층

- OSI 6계층
- 응용 계층이 취급하는 데이터 표현 방식에 맞춰 전송하는 역할 담당
- 통신 장치의 데이터 표현 방식 및 다른 부호 체계 간의 변화 규정
- 프로세스 고유의 표현 방식으로 이뤄진 데이터
-> 표준 데이터 표현방식으로 변환
- 암호화와 데이터 압축

3-2-7. 응용 계층

- OSI 최상의 7계층
- 정보 처리하는 응용 프로그램, 프로세스의 인터페이스와 통신하기 위한 기본적인 응용 기능 제공
- 실제 응용 기능을 지원 -> 사용자가 OSI 통신 환경 사용 가능하도록 함

3-3. 데이터 전송 유형

3-3-1. 전송 방향에 따른 분류

- 1) 단방향 전송: 1대1 통신 환경에서 데이터를 한 방향으로만 전송
- 2) 반이중 전송: 데이터가 양방향으로 전송 but 한번에 한쪽으로만 전송
- 3) 전이중 전송: 양쪽에서 데이터를 전송

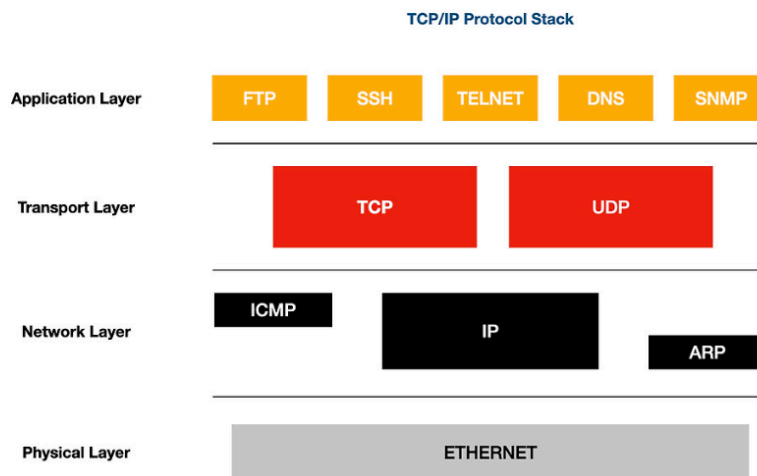
3-3-2. 회선 접속 방식에 따른 분류

- 1) 점대점 전송: 컴퓨터에서 1대1 방식으로 직접 연결
전용 회선 사용 -> 데이터 송수신
- 2) 다지점 전송: 두 개 이상의 통신장치가 하나의 회선 공유

3-4. TCP/IP 프로토콜 스택

3-4-1. TCP와 IP

- TCP: Transport Layer 간의 통신 가능하게하는 규약
- Network Layer 간의 통신을 가능하게 해주는 규약
- 별도의 계층에서 동작하는 프로토콜이지만 대체로 같이 사용함



3-5. 전송 계층의 역할

- 데이터를 받아야할 상대를 정확히 찾아주는 역할
- 실제로 잘 도달하는지 확인함
- 즉, 통신의 신뢰성을 보장하는 역할

3-6. 전송 계층 작동 과정

1) Segmantation

- 패킷은 데이터를 작은 단위로 쪼개서 전송함
- Segmantation은 Application Layer로부터 전달 받은 데이터를 작은 단위로 나누는 과정
- 이 때, 각 Segment에 소스와 목적지의 포트 번호, 데이터의 시퀀스 번호, checksum을 담음
- segment에 왜 포트번호가 담기는가? -> 프로세스마다 다른 포트 번호에서 실행됨 -> 올바른 프로그램을 찾아가기 위해서는 목적지 포트를 알고있어야 하기 때문

2) Error Control

- 수신자가 패킷 조각 하나로 합쳐서 데이터로 구성할 때 유실되었거나 데이터가 손상될 수 있음
- Segment의 Checksum이 손상이나 유실을 확인해줌
- 만약 유실되었으면 해당 부분을 다시 보내주도록 요청

3) Flow Control

- 데이터가 전송되는 양을 컨트롤
- 컴퓨터, 모바일 등 각 장치별 데이터를 프로세싱하는 속도가 다를 수 있음
- 이때 중간에서 프로세싱 속도를 조절해줘야 오류가 발생하지 않음

4. UDP(User Datagram Protocol)란?

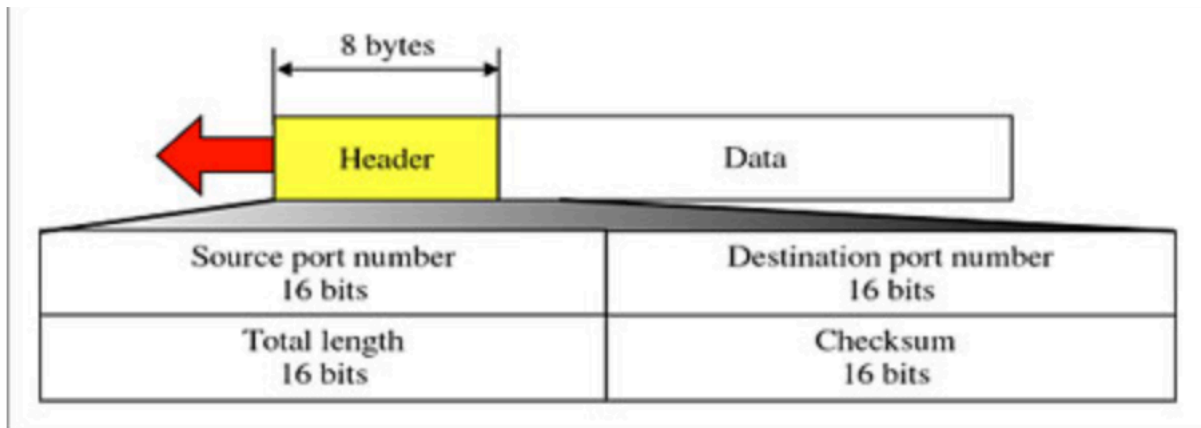
4-1. UDP 개념

- 사용자 데이터그램 프로토콜
- OSI 7계층에서 전송 계층에 해당됨
- 인터넷 상에서 서로 정보를 주고받을 때 정보를 보낸다는 신호나 받는다는 신호 절차 거치지 않음
- TCP와 달리 데이터 수신에 대한 책임을 지지않음
- 신뢰성이 낮음
- 연속성이 좋음

4-2. UDP 특징

- 비연결형, 신뢰성이 없는 전송 프로토콜
- 최소한의 오버헤드만 사용 -> 매우 간단한 프로토콜
- 프로세스 대 프로세스 통신을 제공하는 것 이외에 IP 서비스에 추가되는 기능 없음
- 극히 제한된 오류 점검 기능만을 수행

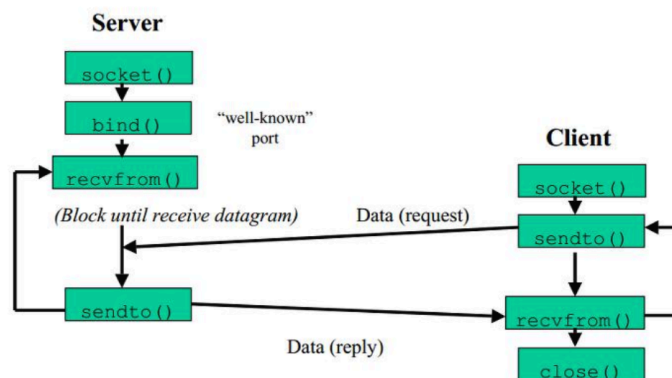
4-3. UDP 헤더 구조



- 발신지 포트 번호: 발신지 호스트 상에서 수행되는 프로세스가 사용하는 포트 번호
- 목적지 포트 번호: 목적지 호스트 상에서 수행되는 프로세스가 사용하는 포트 번호
- 길이: 헤더와 데이터를 합한 사용자 데이터그램의 전체 길이 정의
- 검사합: 사용자 데이터그램에 오류가 있었는지 검사하기 위해 사용
12Byte의 의사 헤더를 추가 -> IP 헤더의 오류로 인한 잘못된 전달 방지

4-4. UDP 데이터그램의 전송 과정

UDP Client-Server



UDP 흐름

1) Socket()

- 통신 시 endpoint 를 만들어주는 함수

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

- domain: 요청된 주소 도메인, IPv4 나 IPv6같은 프로토콜 집합
- type: 생성된 소켓 유형 -> UDP인 경우 SOCK_DGRAM을 1매개변수로 사용
- protocol: UDP나 TCP같은 특정 프로토콜을 명시

2) bind()

- socket()으로 만들어진 소켓에 로컬 이름 붙여줌

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- sockfd: socket() 호출에서 반환된 소켓 descriptor
- const struct sockaddr: socket에 붙여줄 이름을 포함하는 sockaddr 구조체의 주소
- addrlen: 두번째 인자인 sockaddr로 생성한 주소의 크기
- bind는 왜 필요한가? -> 꼭 사용할 필요는 없음
-> OS가 패킷을 어디로 전달하는지 모를 때 사용하면됨

3) recvfrom()

- 소켓 descriptor(소켓 fd)로부터 메시지를 수신해서 버퍼에 저장

```
#include <sys/types.h>
#include <sys/socket.h>

ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t
                 *addrlen);
```

- int sockfd: socket() 호출에서 반환된 소켓 fd
- void *buf: 데이터를 받을 버퍼에 대한 주소
- size_t len: 두번째 인자인 buf의 길이
- int flag: 데이터를 읽을 때에 주는 옵션
- struct sockaddr *src_addr: 데이터가 수신되는 소켓 주소 struct에 대한 주소
- socklen_t *addrlen: 5번째 인자인 src_addr의 크기

4) sendto()

- 소켓 fd를 통해 수신자에게 데이터를 보내는 함수

```
#include <sys/types.h>
#include <sys/socket.h>

ssize_t sendto(int sockfd, const void *buf, size_t len, int
flags,
               const struct sockaddr *dest_addr, socklen_t
addrlen);
```

- int sockfd: socket() 호출에서 반환된 소켓 fd
- const void *buf: 전송할 메시지가 포함된 버퍼에 대한 주소
보낼 메시지는 변경되면 안되기 때문에 const 붙음
- size_t len: 두번째 인자인 buf의 길이
- int flag: 데이터를 읽을 때에 주는 옵션
- struct sockaddr *dest_addr: 데이터를 받을 호스트의 주소
- socklen_t *addrlen: 5번째 인자인 src_addr의 크기

5) close()

- 소켓 fd를 close하는 함수

```
#include <unistd.h>

int close(int fd);
```

- int fd: 닫을 socket fd, 대기 중인 입력 데이터가 있을 때는 완전히 닫히지 않을 수 있음

4-5. UDP 포트번호

- 0~1023: well-known ports
-> 대부분의 중요한 프로토콜 용도
- 1024~49151: registered ports
-> IANA에 의해 사용처가 등재된 포트
- 49152~65535: dynamic ports
-> 클라이언트용 단기 포트

4-6. UDP 장점

- 1) 단순함
 - 작은 헤더 크기 : UDP = 8바이트 헤더 / TCP = 20바이트 헤더
- 2) 무상태
 - 메모리 효율성: 연결별 상태 정보 저장 X
 - 확장성: 상태 관리 부담없이 쉽게 스케일링 가능
 - 리소스 절약: 연결이 많아져도 메모리 사용량 증가폭 적음
- 3) 낮은 지연 시간
 - 핸드 셰이크 없음: 즉시 데이터 전송 가능
 - 사전 통신 불필요: 연결 수립 과정 생략
 - 실시간 애플리케이션: 게임, 음성, 영상 통화에 최적

4-7. UDP 단점

- 1) 전송 보장 없음
 - 확인 불가: 서버가 데이터를 받았는지 모름
 - 상태 불확실: 데이터 베이스 업데이트 여부 확인 불가
 - 신뢰성 부족: 중요한 데이터 전송에는 부적합
- 2) 순서 보장 없음
 - 패킷 1,2,3 순서대로 보내도 순서대로 도착한다는 보장 X
 - 애플리케이션에서 순서 복원 로직 구현 필요
 - 파일 전송 문제점 존재
- 3) 보안 취약
 - UDP 플러딩 공격, DNS 증폭 공격 등 사전 인증이나 연결 수립 과정이 없어 보안에 취약함